

WEB API & ANGULAR CON VISUAL STUDIO .NET CORE

Capítulo 2: Implementación de una arquitectura de
Software

Four empty circles arranged vertically on the right side of the slide.



OBJETIVOS

- Explicar la estructura de una arquitectura limpia.
- Diferenciar los patrones mediator y dependency injections.
- Utilizar bibliotecas como MediatR, FluentValidation y AutoMapper.





AGENDA

1. Desarrollo de software basado en Domain Driven Design.
2. Clean Architecture.
3. Uso del patrón Command Query Responsibility Segregation (CQRS).
4. Modelando el dominio de la aplicación.
5. Creando la capa de aplicación.
6. Creando la capa de infraestructura.
7. Objetos de transferencia de datos.
8. Uso de Dependency Injection.
9. Uso de Mediator com MediatR, FluentValidation y AutoMapper





1. DESARROLLO DE SOFTWARE BASADO EN DOMAIN DRIVEN DESIGN

Domain-Driven Design (DDD) es una metodología de desarrollo de software basada en modelos de dominio. Fue acuñada por Eric Evans en 2003 y está diseñada para ayudar a los desarrolladores a crear aplicaciones de software de alto nivel.

DDD se centra en la comprensión profunda de un dominio específico para el desarrollo de aplicaciones de software. Esto significa que el enfoque de DDD es dirigido principalmente por el dominio y se centra en el negocio.





2. CLEAN ARQUITECTURE

Clean Architecture o arquitectura limpia es un compendio de principios y patrones de desarrollo que tienen como objetivo el facilitar el proceso de construcción del software, así como su mantenimiento.

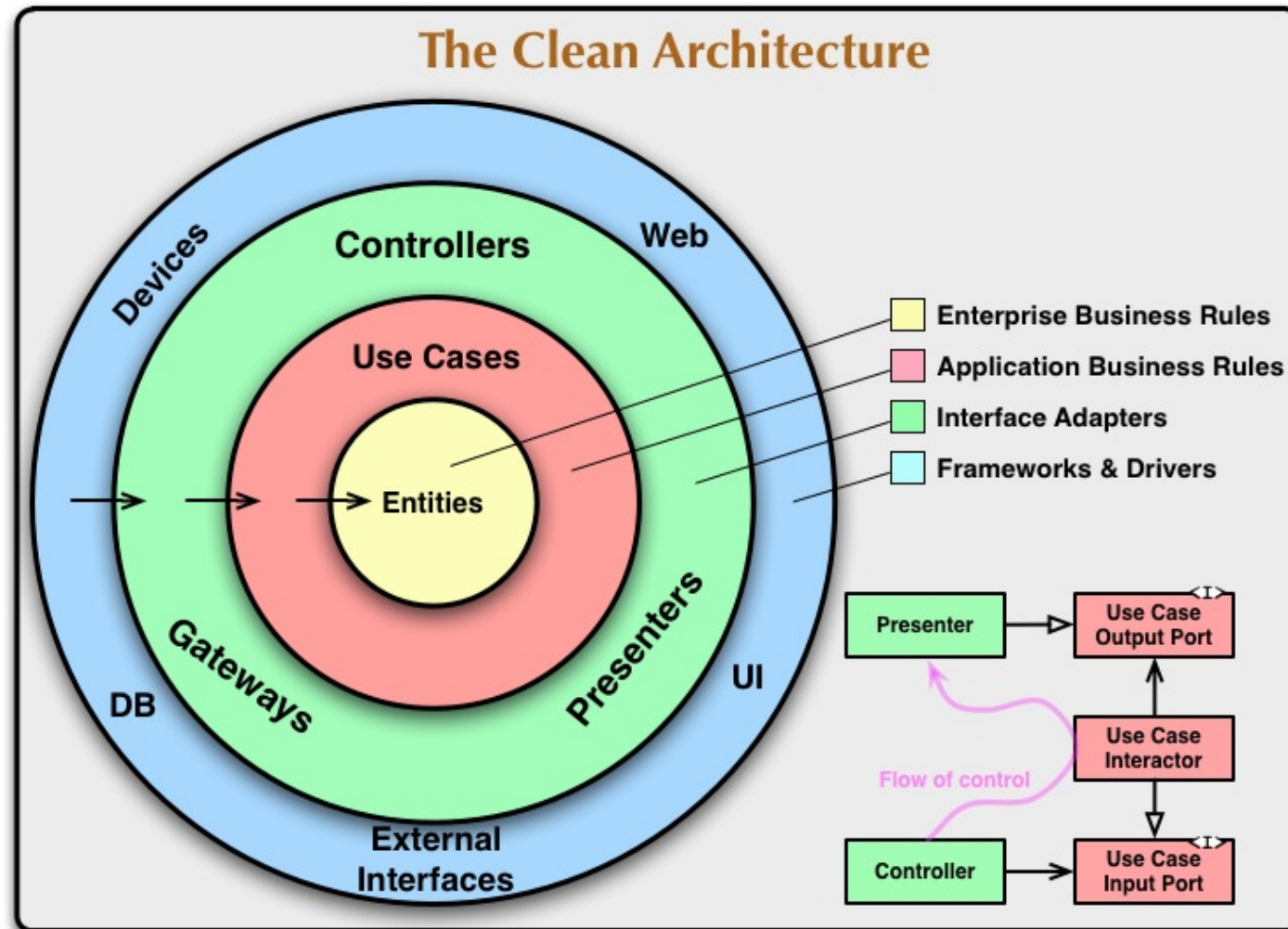
Beneficios

- Creación de aplicaciones desacopladas que son más fáciles de probar.
- Mayor flexibilidad para añadir o remover funcionalidades del software.
- Diseño basado en componentes con responsabilidades bien definidas.





2. CLEAN ARQUITECTURE

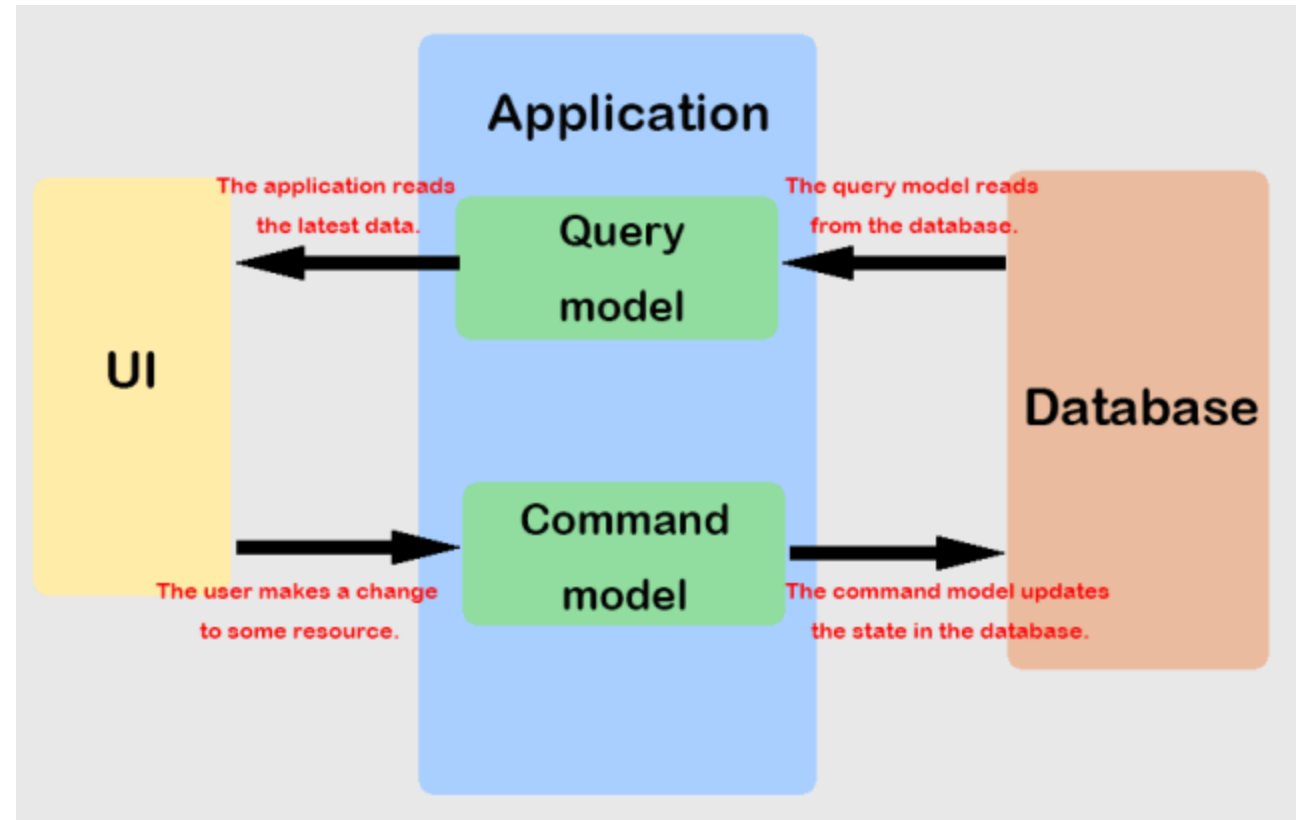




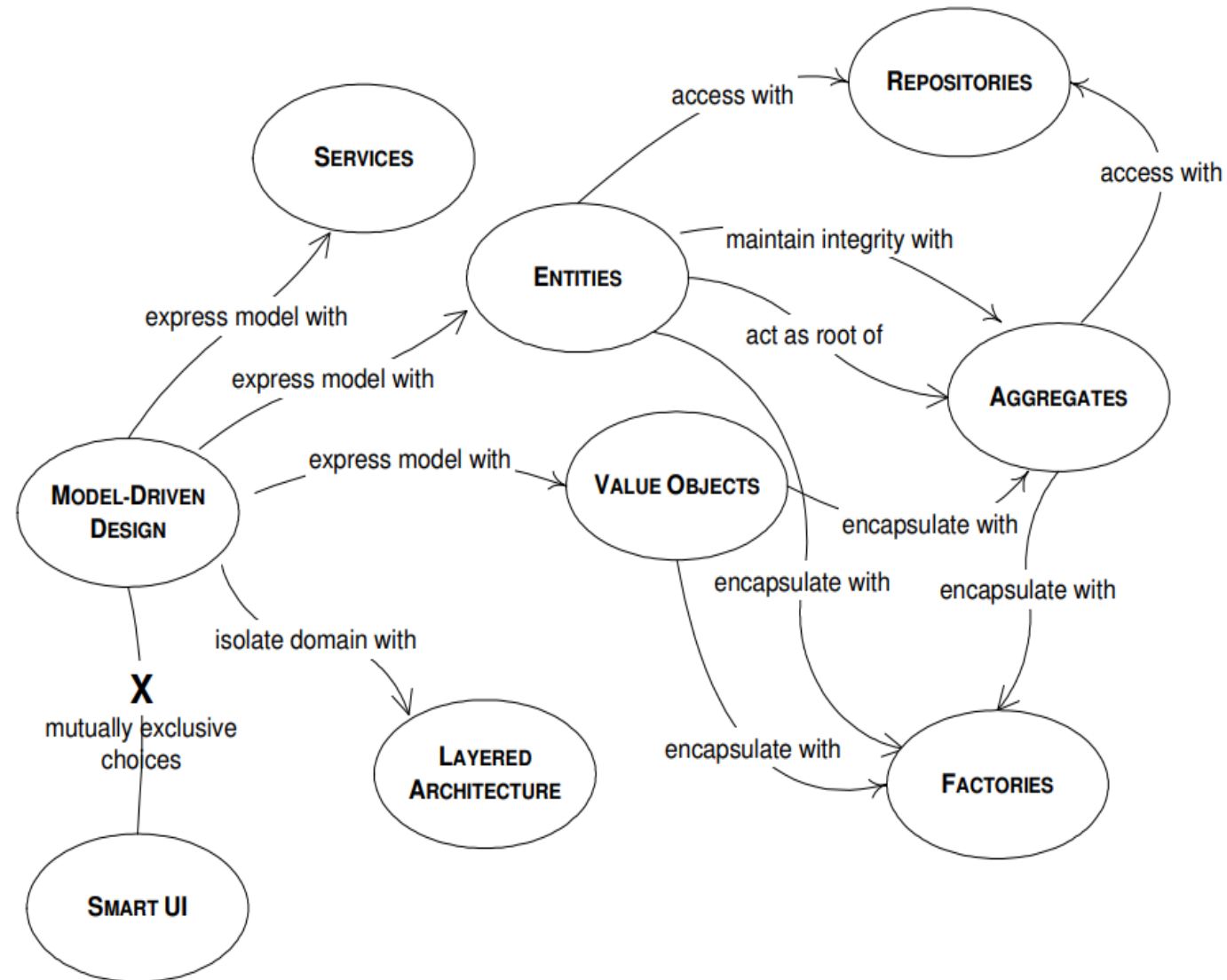
3. USO DEL PATRÓN COMMAND QUERY RESPONSIBILITY SEGREGATION (CQRS)

Patrón para diseñar software que permite separar y segregar las escrituras de las lecturas de una base de datos.

Entiende las escrituras como una acción; por ejemplo, de registrar un usuario y las lecturas como una petición o ver usuarios.



4. MODELANDO EL DOMINIO DE LA APLICACIÓN

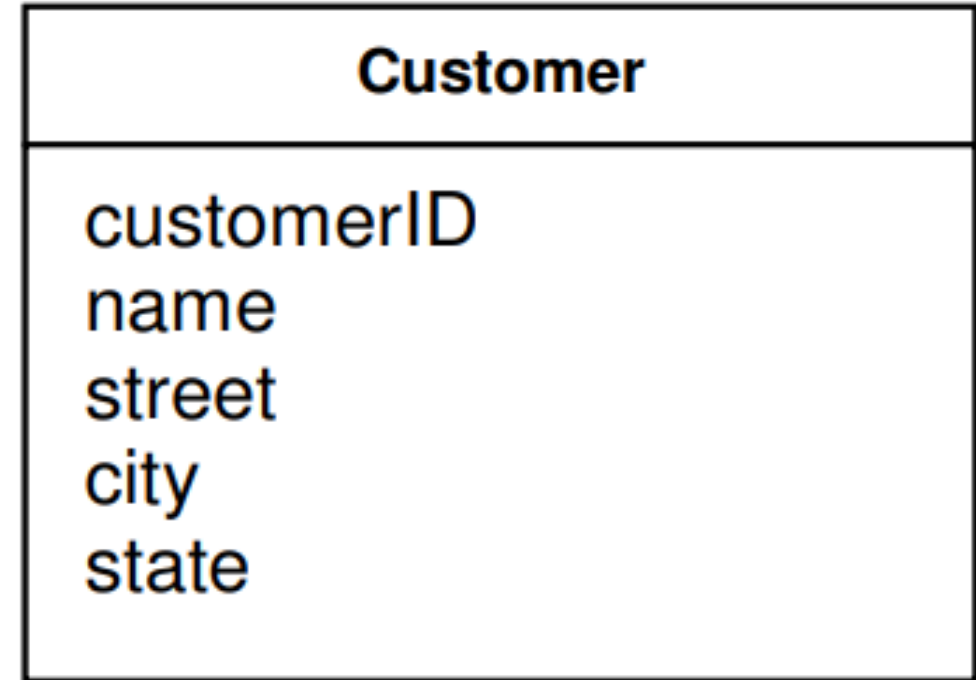




4. MODELANDO EL DOMINIO DE LA APLICACIÓN

Entidades de dominio – Domain Entities

Categoría de objetos que tienen una identidad, que permanece igual en todos los estados del software. Para estos objetos no son los atributos lo que importa, deberían ser consideradas desde el inicio del proceso de modelado.

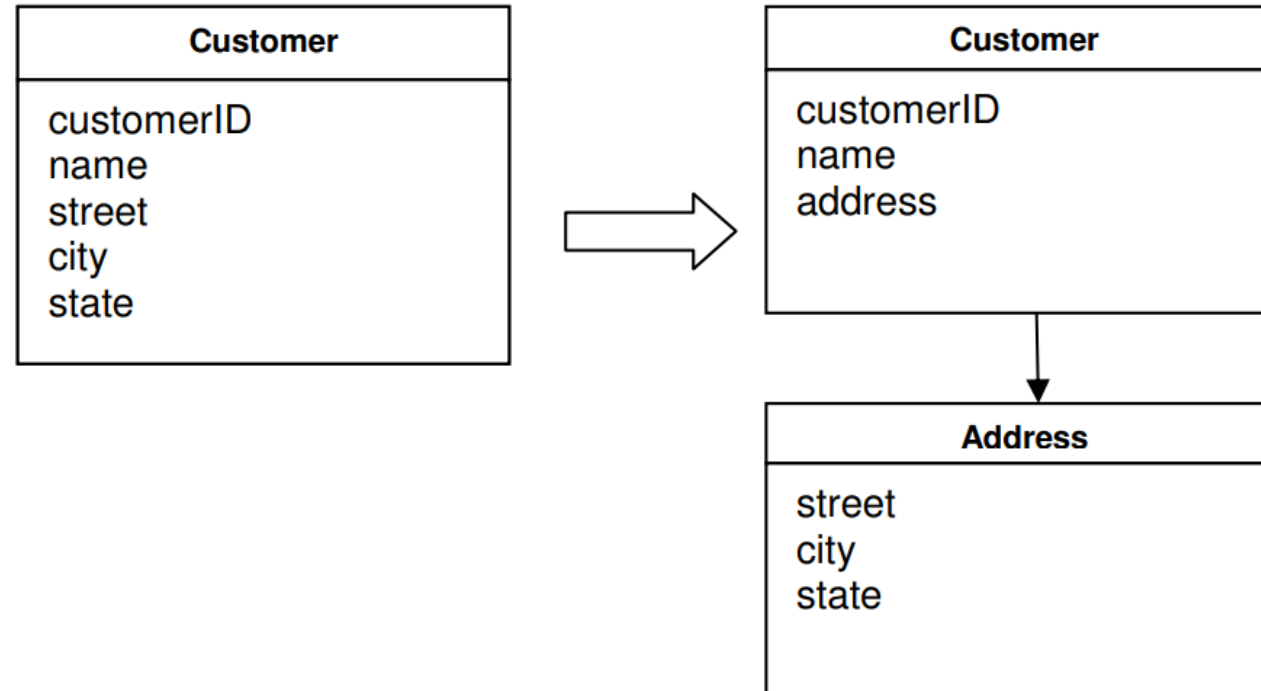




4. MODELANDO EL DOMINIO DE LA APLICACIÓN

Objetos de valor – Value Objects

Categoría de objetos cuya identidad está representado por sus atributos.

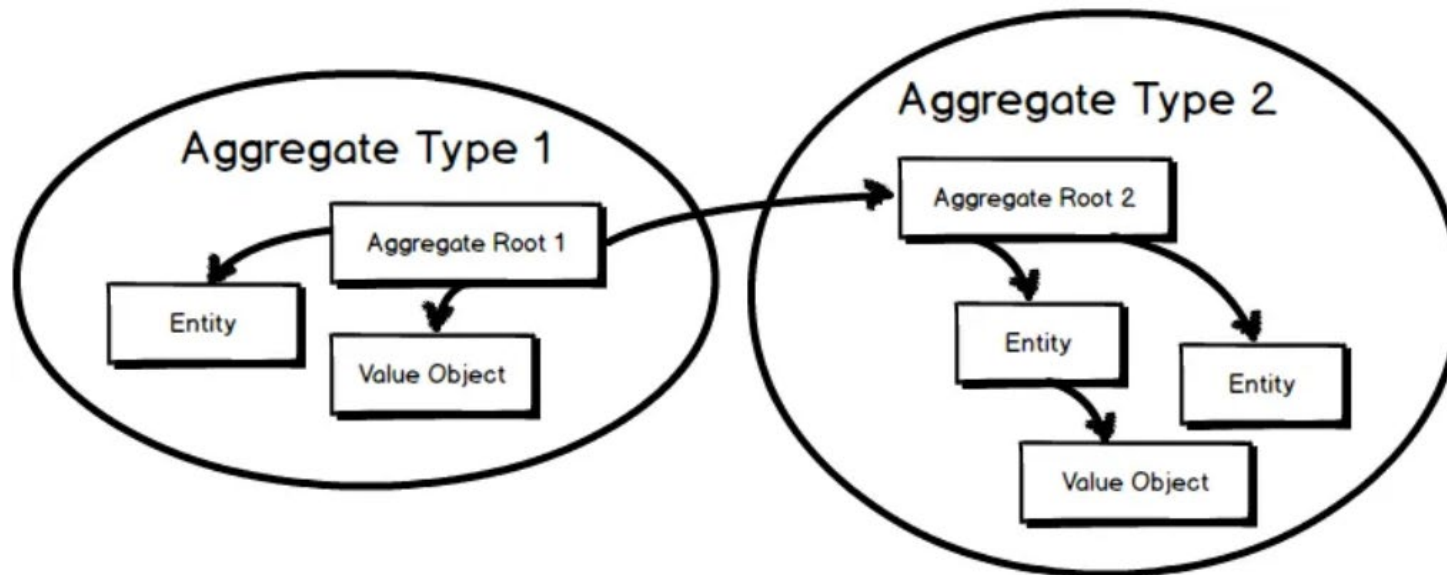




4. MODELANDO EL DOMINIO DE LA APLICACIÓN

Objetos agregados – Aggregates

Corresponden a un grupo de objetos asociados, los cuales son considerados como una unidad con respecto a los cambios de la información.



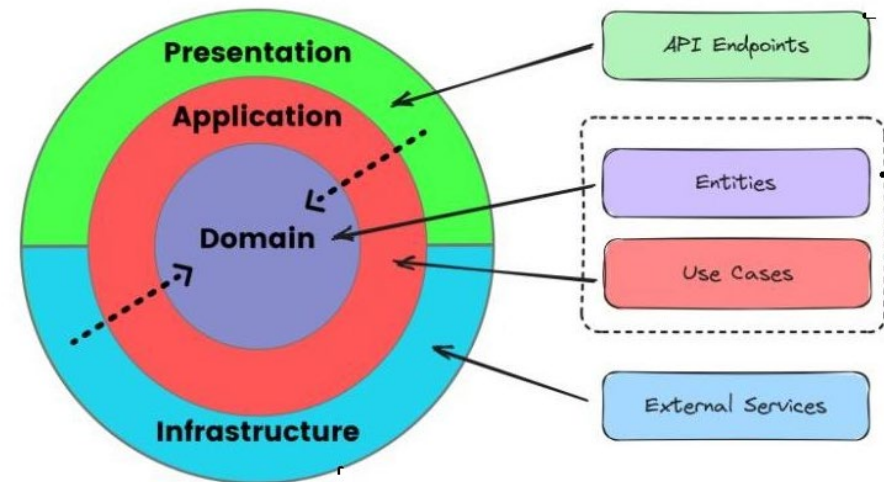


5. CREANDO LA CAPA DE APLICACIÓN

En esta capa se implementan los casos de uso y se integran los modelos del dominio, junto con los servicios del dominio y los repositorios. En esta capa se podría implementar las validaciones de los datos de entrada y también se define la exposición de los casos de uso, mediante algún patrón como mediator o CQRS.

¡Importante!

Esta capa es orientada al negocio y no debe estar relacionado a una tecnología.





6. CREANDO LA CAPA DE INFRAESTRUCTURA

Esta es una capa relacionada con la tecnología que será usada en la aplicación. Por ejemplo, en este nivel se definen, en qué medio se van a guardar los datos de la aplicación, con qué base de datos, qué biblioteca de acceso a datos será usada.

Tareas que podrías realizar:

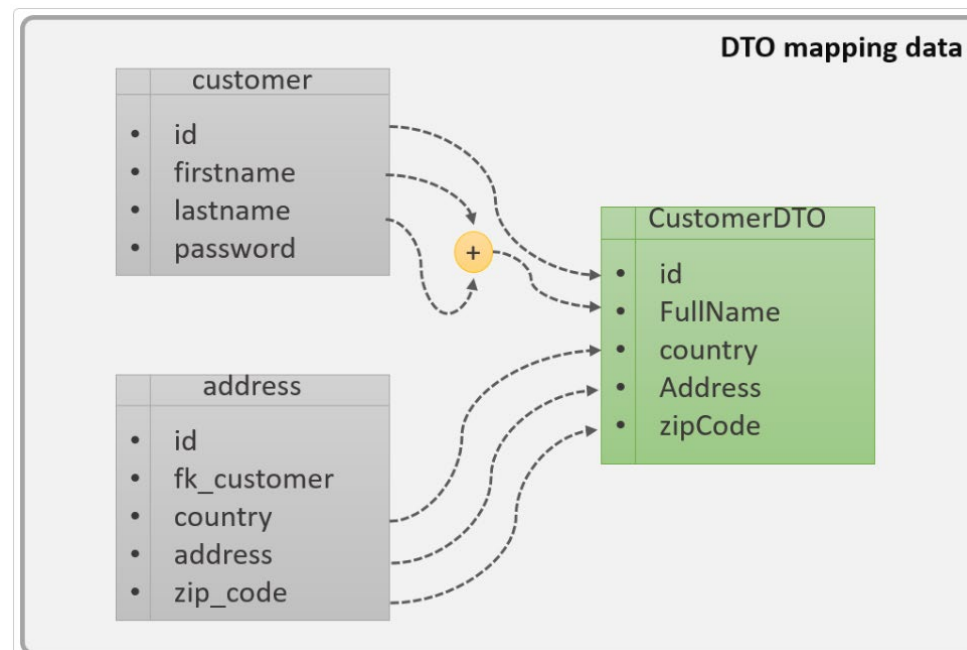
- Implementación de los repositorios.
- La comunicación con otros servicios web.
- Comunicación con algún servicio de mensajería.
- Integración con algún proveedor de envío de SMS, e-mail o WhatsApp





7. OBJETOS DE TRANSFERENCIA DE DATOS

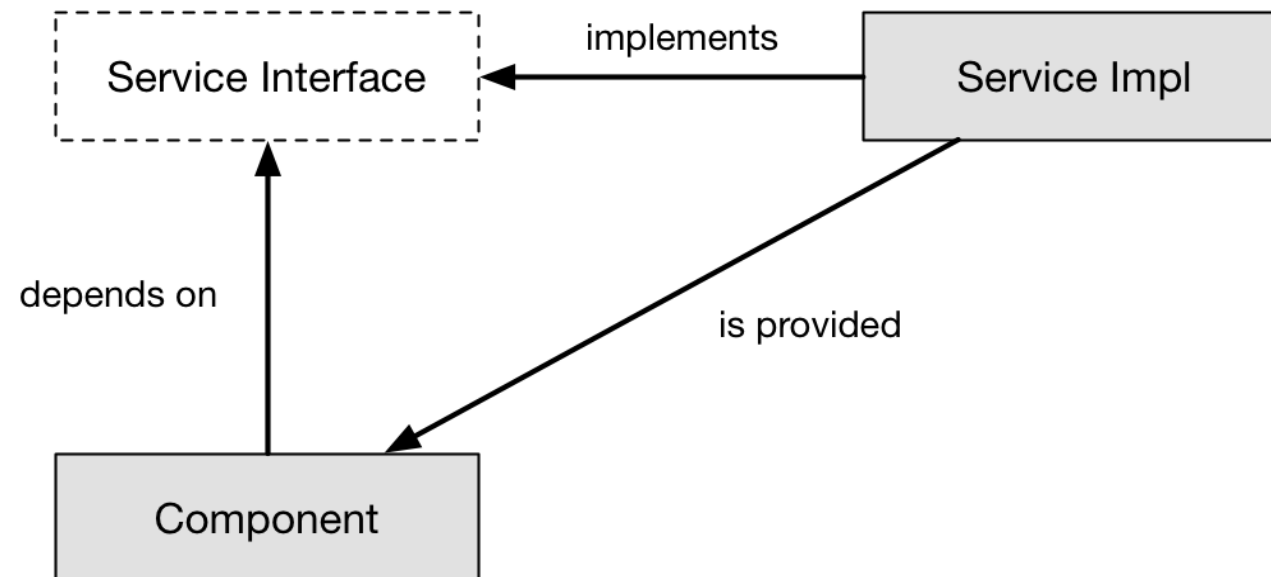
El patrón DTO tiene como finalidad, crear un objeto plano (POJO) con una serie de atributos que puedan ser enviados o recuperados del servidor en una sola invocación. De tal forma que, un DTO puede contener información de múltiples fuentes o tablas, y concentrarlas en una única clase simple.





8. USO DE DEPENDENCY INJECTION - DI

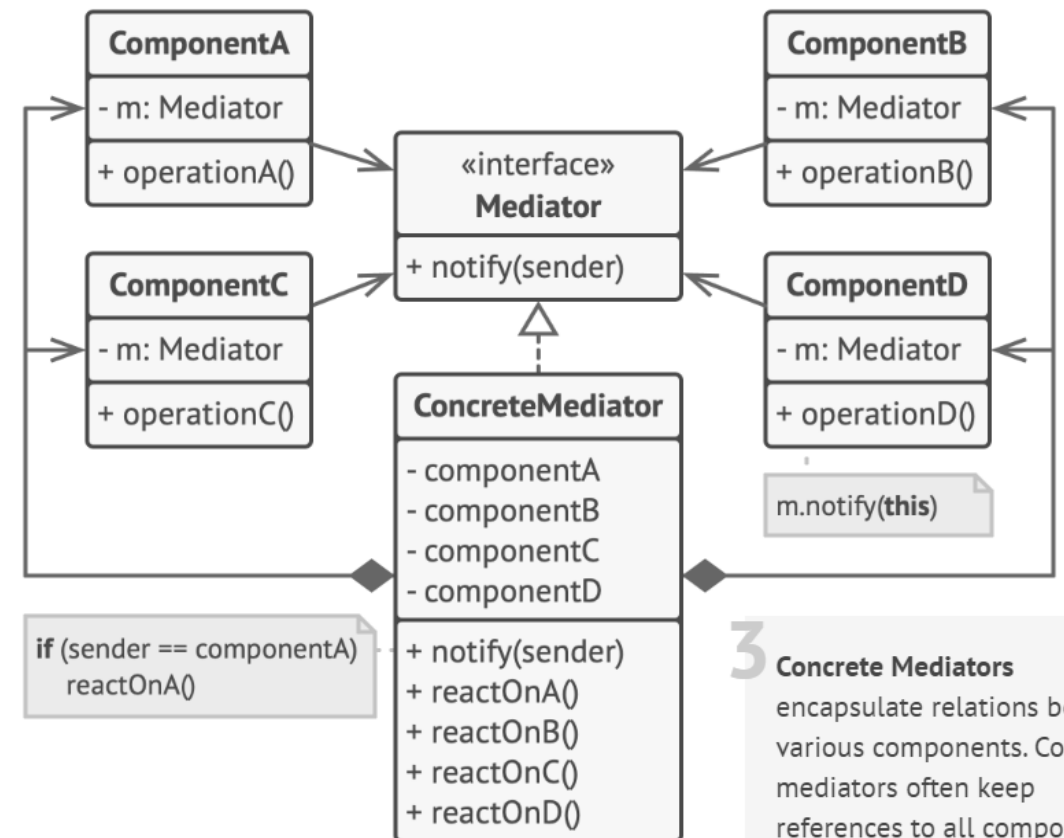
Este patrón de diseño permite evitar la dependencia entre los componentes de software, haciendo que dependan de abstracciones y no, de implementaciones directas. Permitiendo bajar el acoplamiento entre los componentes del sistema; facilitando el mantenimiento y evolución del sistema.





9. USO DE MEDIATOR CON MEDIATR, FLUENTVALIDATION Y AUTOMAPPER

Mediator: el patrón restringe las comunicaciones directas entre los objetos y los obliga a colaborar solo a través de un objeto mediador. La biblioteca MediatR implementa el patrón mediator y permite integrarlo en un proyecto de .NET



3 Concrete Mediators
encapsulate relations between various components. Concrete mediators often keep references to all components they manage and sometimes even manage their lifecycle.



9. USO DE MEDIATOR CON MEDIATR, FLUENTVALIDATION Y AUTOMAPPER

```
public class CustomerValidator : AbstractValidator<Customer>
{
    public CustomerValidator()
    {
        RuleFor(x => x.Surname).NotEmpty();
        RuleFor(x => x.Forename).NotEmpty().WithMessage("Please specify a first name");
        RuleFor(x => x.Discount).NotEqual(0).When(x => x.HasDiscount);
        RuleFor(x => x.Address).Length(20, 250);
        RuleFor(x => x.Postcode).Must(BeAValidPostcode).WithMessage("Please specify a valid postcode");
    }

    private bool BeAValidPostcode(string postcode)
    {
        // custom postcode validating logic goes here
    }
}
```

Fluent Validation: biblioteca en .NET que permite implementar validaciones de datos utilizando expresiones lambda.

<https://docs.fluentvalidation.net/en/latest/>





9. USO DE MEDIATOR CON MEDIATR, FLUENTVALIDATION Y AUTOMAPPER

AutoMapper: permite implementar el mapping entre dos objetos, pasando la información de manera automática de cada de una de las propiedades de un objeto para otro objeto.

```
public class Source
{
    public int SomeValue { get; set; }
}

public class Destination
{
    public int SomeValuefff { get; set; }
}
```

```
var configuration = new MapperConfiguration(cfg =>
    cfg.CreateMap<Source, Destination>());

configuration.AssertConfigurationIsValid();
```





LABORATORIO Nº 1: APLICACIÓN ASP.NET WEB API Y MODELO DE ARQUITECTURA LIMPIA

En este laboratorio, crearás una estructura de un proyecto web API. Además, podrás estructurar el proyecto utilizando un modelo de arquitectura limpia e integrar las bibliotecas FluentValidation, MediatR y AutoMapper.





TAREA Nº 1: MODELO PARA LA APLICACIÓN

Al finalizar la tarea, lograrás:

- Utilizar una estructura de arquitectura limpia y sus componentes relacionados.





LECTURAS ADICIONALES

Para obtener información, puede consultar:

- Blancarte, O. ([30 noviembre, 2018](#)). Data Transfer Object (DTO) – Patrón de diseño. Oscar Blancarte. <https://www.oscarblancarteblog.com/2018/11/30/data-transfer-object-dto-patron-diseno/>
- Refactoring Guru. (s.f.). Mediator. <https://refactoring.guru/design-patterns/mediator>
- Šahbaz, E. (01 de julio de 2023). *Exploring the Power of Aggregates in Domain-Driven Design and Clean Architecture*. Medium. <https://medium.com/@edin.sahbaz/exploring-the-power-of-aggregates-in-domain-driven-design-and-clean-architecture-6408d6128d3b>





RESUMEN

- DDD se centra en la comprensión profunda de un dominio específico para el desarrollo de aplicaciones de software.
- La arquitectura limpia, permite facilitar el proceso de construcción del software, así como su mantenimiento.



